

# Onboarding de transportadoras: perfis dinâmicos, isolamento por schema e convites

Como a RookHub passa a receber, aprovar e entregar um ambiente pronto — conectado à telemetria, com a marca do cliente e os módulos contratados — e como cada Dono ganha autonomia para desenhar os cargos e montar o próprio time.

---

**Data**

11/09/2026

**Escopo**

Backend-web · System-web · Infra

**Situação**

Aguardando decisão da equipe

**Este documento é uma proposta, não uma decisão.** Nada foi implementado. O objetivo é que a equipe avalie o desenho — em especial a substituição dos perfis fixos por cargos customizáveis e a mudança de isolamento de dados — antes de qualquer linha de código.

# 1. Por que mexer nisso agora

---

A RookHub vai deixar de ser um sistema de um cliente só. Pessoas da empresa parceira já pedem acesso, e novas transportadoras chegam depois do MVP. Hoje **não existe nenhum caminho para criar acesso**:

- `users` e `tenants` são somente leitura no Java: sem *setters*, sem endpoint de escrita.
- Não há tabela de convites, nem qualquer transporte de e-mail no projeto.
- Em produção o banco nasce vazio — nem um `SUPER_ADMIN` consegue logar.
- As telas `/convite/:token`, `/admin-saas/*` e `/gestao/equipe` existem, mas são simuladas.
- Os perfis são um **enum fixo de 6 valores**, repetido no banco e no frontend. Não existe "Diretor" — e não há como criar um.

O resultado pretendido é um fluxo fechado:

1. A transportadora pede acesso pelo site institucional.
2. O time da RookHub é avisado por e-mail e decide no backoffice.
3. A aprovação provisiona a empresa e cria **apenas a credencial do Dono**.
4. O Dono define os cargos da própria empresa e cadastra o time sozinho.

## Decisões já tomadas

| Tema                        | Decisão                                                                            |
|-----------------------------|------------------------------------------------------------------------------------|
| Isolamento de dados         | Schema por tenant, num único Postgres                                              |
| Vínculo usuário ↔ empresa   | 1:1. A mesma pessoa em duas empresas tem duas credenciais                          |
| Perfis do cliente           | Cargos customizáveis: o Dono cria, nomeia e escolhe as permissões                  |
| Equipe RookHub              | Dois papéis fixos: TI Topo e TI Operacional                                        |
| Suporte (TI Operacional)    | Leitura dos dados do cliente, todo acesso auditado                                 |
| Exclusão de conta           | Desativar na hora + anonimizar depois. Nunca DELETE                                |
| Quem cria e desliga         | TI Topo: contas da plataforma e o Dono. Dono: a equipe dele                        |
| Telemetria                  | MiX conecta na hora; outro fornecedor entra como pendente e não trava o onboarding |
| Conta no fornecedor         | É do cliente. A RookHub intermedia, não revende                                    |
| Marca                       | Logo e cores pelo TI Topo; fonte de lista homologada. Aparece já no login          |
| Planos                      | O plano barra no backend e na tela. É o teto; o cargo distribui o que sobrou       |
| Notificação / e-mail        | Resend (HTTP)                                                                      |
| Entrega da credencial       | Link de convite por e-mail, token de uso único                                     |
| Formulário de pedido        | Site institucional → endpoint público na API                                       |
| Subdomínio                  | Slug definido pelo time da RookHub na aprovação                                    |
| Bootstrap do primeiro admin | Variáveis de ambiente, idempotente no boot                                         |
| Hospedagem do SPA           | Cloudflare Pages; domínio por tenant registrado via API                            |
| Dados atuais                | Migração in-place de public → tenant_servioeste                                    |

## Duas medições que definem o tamanho da obra

### 1. A tenancy não se espalha pelo código

As colunas `tenant_id` e os `WHERE tenant_id = :tenant` continuam como estão. São 136 ocorrências em 22 arquivos Java. Movê-las seria reescrever o backend sem ganho: o schema já isola, e o `WHERE` vira defesa em profundidade de graça. O único código de tenancy que muda é *como a conexão escolhe o schema* — um ponto só, na Fase 3.

### 2. A autorização também não

O backend inteiro tem **19 @PreAuthorize** em **3 controllers** (`FleetController` 10, `MixDiagnosticsController` 7, `VoiceController` 2), mais uma checagem solta em `FleetContextBuilder`. Trocar `hasRole` por `hasAuthority` é trabalho mecânico de horas. No frontend são 50 usos em 19 arquivos, mas quase todos já passam por `hasPermission()` — a abstração certa já existe, muda só quem a alimenta.

## Por que schema, e não banco físico por cliente

Banco físico por transportadora entrega isolamento máximo, mas o custo operacional é alto: 29 migrations rodando em N bancos a cada deploy, N pools de conexão, backup em laço e — o ponto mais duro — o painel de Super Admin deixaria de conseguir um único `SELECT` para métricas globais. Schema dentro do mesmo Postgres entrega a garantia que importa (nenhuma consulta atravessa clientes, mesmo com bug de `WHERE`) por uma fração do custo.

Há um detalhe específico desta stack: `daily_vehicle_metrics` é um *continuous aggregate* do TimescaleDB, e `max_background_workers` é um limite **por instância**, não por schema. Com banco por cliente isso viraria problema real de capacidade; com schema, não.

## 2. O modelo de perfis

Esta é a parte mais nova e a de maior alcance. Hoje existe um enum de seis valores ( `OWNER`, `MANAGER`, `OPERATOR`, `MAINTENANCE`, `SUPER_ADMIN`, `DRIVER` ) repetido em três lugares: o enum Java, o `CHECK` do banco e o mapa `ROLE_PERMISSIONS` do frontend. Ele é substituído por dois modelos diferentes, um para cada lado da plataforma.

### Lado RookHub — papéis fixos

São nossos. Não há por que torná-los configuráveis.

| Papel                         | Quem é               | O que pode                                                                                                                                           |
|-------------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PLATFORM_ADMIN</code>   | TI Nível Topo        | Controle global. <b>Único</b> que cria e desliga contas — as da plataforma e o Dono de cada transportadora. Aprova solicitações e suspende empresas. |
| <code>PLATFORM_SUPPORT</code> | TI Nível Operacional | Suporte e manutenção. <b>Leitura</b> dos dados de qualquer cliente, sempre auditada. Nenhuma escrita e nenhuma gestão de conta.                      |

### Lado cliente — cargos que o Dono desenha

O cargo deixa de ser enum e vira dado da transportadora. O Dono cria "Diretor Financeiro", marca quais permissões ele tem e atribui a pessoas — sem deploy nosso.

```
CREATE TABLE roles (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id   UUID NOT NULL REFERENCES tenants (id),  
  key         TEXT NOT NULL,          -- 'dono', 'diretor-financeiro'  
  name        TEXT NOT NULL,          -- 'Diretor Financeiro'  
  description TEXT,  
  permissions TEXT[] NOT NULL DEFAULT '{}',  
  landing     TEXT NOT NULL DEFAULT 'operacional', -- 'gestao' | 'operacional'  
  -- Cargo de sistema: semeado por nós, não pode ser apagado nem renomeado.  
  -- O Dono é o único com system = TRUE.  
  system      BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  CONSTRAINT roles_key_unique UNIQUE (key)  
);  
  
ALTER TABLE users ADD COLUMN role_id UUID REFERENCES roles (id);
```

Toda transportadora nova nasce com seis cargos semeados:

| Cargo      | Sistema | Entra em | Observação                                                     |
|------------|---------|----------|----------------------------------------------------------------|
| Dono       | ✓       | /gestao  | Todas as permissões, mais gestão de equipe e cargos. Indelével |
| Gestor     | —       | /gestao  | Editável e apagável pelo Dono                                  |
| Diretor    | —       | /gestao  | <b>Novo.</b> Visão gerencial sem gestão de equipe              |
| Operador   | —       | /app     | Mantém a regra RF-007 de não ver financeiro                    |
| Manutenção | —       | /app     |                                                                |
| Motorista  | —       | /app     | Só o aplicativo do motorista                                   |

Os cinco não-sistema nascem apenas como sugestão: o Dono renomeia, altera permissões, apaga ou cria outros do zero.

### Quem pode o quê

| Ação                                  | TI Topo              | TI Oper.   | Dono | Demais cargos    |
|---------------------------------------|----------------------|------------|------|------------------|
| Criar/desligar conta da plataforma    | ✓                    | —          | —    | —                |
| Aprovar transportadora e criar o Dono | ✓                    | —          | —    | —                |
| Criar/editar/apagar cargos da empresa | intervenção auditada | —          | ✓    | —                |
| Convidar/desligar membro da equipe    | intervenção auditada | —          | ✓    | —                |
| Ler dados operacionais do cliente     | ✓ auditado           | ✓ auditado | ✓    | conforme o cargo |
| Escrever dados operacionais           | —                    | —          | ✓    | conforme o cargo |

**Suposição que precisa de confirmação.** O texto diz que o Dono é o *responsável exclusivo* por gerir a equipe. Então gestão de equipe e gestão de cargos ficam **não delegáveis**: pertencem só ao cargo Dono e nem aparecem no editor de cargos. Se um dia ele precisar delegar isso a um Diretor, basta liberar as duas permissões no catálogo — a estrutura já suporta.

### 3. Fase 1 — Plano de controle: schema `platform`

Nova pasta de migrations `db/migration/platform/`, executada por uma instância Flyway própria. É aqui que vive tudo que é da plataforma e não de nenhum cliente: empresas, contas da equipe RookHub, solicitações de acesso, fila de e-mail e auditoria.

```
CREATE SCHEMA IF NOT EXISTS platform;

CREATE TABLE platform.tenants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        TEXT NOT NULL UNIQUE,    -- subdomínio E nome do schema
  schema_name TEXT NOT NULL UNIQUE,
  name        TEXT NOT NULL,
  document    TEXT,
  plan        TEXT NOT NULL DEFAULT 'business',
  status      TEXT NOT NULL DEFAULT 'trial',
  trial_ends_at    TIMESTAMPTZ,
  provisioning_state TEXT NOT NULL DEFAULT 'PENDING', -- PENDING|RUNNING|READY|FAILED
  provisioning_error TEXT,
  active      BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT tenants_slug_format CHECK (slug ~ '^[a-z][a-z0-9-]{2,30}$'),
  CONSTRAINT tenants_plan_check   CHECK (plan IN ('starter','business','enterprise')),
  CONSTRAINT tenants_status_check CHECK (status IN ('active','trial','suspended','cancelled'))
);

-- Equipe interna da RookHub. Papéis FIXOS.
CREATE TABLE platform.platform_users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  name        TEXT NOT NULL,
  role        TEXT NOT NULL,
  must_change_password BOOLEAN NOT NULL DEFAULT FALSE,
  active      BOOLEAN NOT NULL DEFAULT TRUE,
  deactivated_at TIMESTAMPTZ,
  anonymized_at TIMESTAMPTZ,
  created_by   UUID REFERENCES platform.platform_users (id),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT platform_users_role_check
    CHECK (role IN ('PLATFORM_ADMIN','PLATFORM_SUPPORT'))
);
```

Mais `platform.access_requests` (a fila de solicitações), `platform.email_outbox` (fila de envio, para o convite não se perder se a Resend falhar) e `platform.audit_log`. Slugs reservados, como constante Java: `app`, `api`, `www`, `admin`, `mail`, `cdn`, `status`, `docs`, `dev`, `staging`, `painel`, `suporte`.

## 4. Fase 2 — Migrations por tenant, cargos e provisionamento

**Mover** `db/migration/V1..V27` para `db/migration/tenant/` **sem editar nada**. Dentro do schema do cliente os `tenant_id` são inofensivos, e reescrever 27 arquivos não pagaria. Acrescenta-se a V28, que cria a tabela de cargos mostrada na seção 2, mais:

```
-- V28, continuação
ALTER TABLE users ADD COLUMN must_change_password BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE users ADD COLUMN invited_by      UUID REFERENCES users (id);
ALTER TABLE users ADD COLUMN last_login_at   TIMESTAMPTZ;
ALTER TABLE users ADD COLUMN deactivated_at   TIMESTAMPTZ;
ALTER TABLE users ADD COLUMN deactivated_by   UUID REFERENCES users (id);
ALTER TABLE users ADD COLUMN anonymized_at   TIMESTAMPTZ;

-- O e-mail passa a ser único DENTRO da empresa, não no mundo.
-- Anonimizados saem do índice, então o endereço pode ser reusado.
ALTER TABLE users DROP CONSTRAINT users_email_unique;
CREATE UNIQUE INDEX users_email_unique ON users (lower(email)) WHERE anonymized_at IS NULL;

CREATE TABLE invites (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES tenants (id),
  user_id     UUID NOT NULL REFERENCES users (id) ON DELETE CASCADE,
  token_hash  TEXT NOT NULL UNIQUE,    -- SHA-256; o token cru só existe no e-mail
  expires_at  TIMESTAMPTZ NOT NULL,
  accepted_at TIMESTAMPTZ,
  revoked_at  TIMESTAMPTZ,
  created_by  UUID,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Uma **V29** fecha a transição: migra `users.role` para o `role_id` do cargo equivalente, torna a coluna obrigatória e **derruba** o `CHECK` de papel e a coluna de texto.

### Catálogo de permissões

As 21 permissões que já existem em `System-web/src/types/auth.ts` viram a fonte canônica, espelhadas num enum em Java e servidas por `GET /v1/permissions` — com chave, rótulo e grupo — para o editor de cargos montar os checkboxes sem nada fixo no código da tela. Entram quatro novas: gestão de equipe e gestão de cargos (não delegáveis) e, do lado da plataforma, gestão de contas e leitura de suporte.

### TenantProvisioningService

Ao aprovar uma transportadora:

1. valida o slug (formato, reservados, unicidade);
2. `CREATE SCHEMA tenant_<slug>;`
3. roda o Flyway de `db/migration/tenant` apontado para esse schema;
4. insere a linha espelho em `tenant_<slug>.tenants` com o **mesmo UUID** de `platform.tenants` — é o que mantém os 136 `WHERE tenant_id` funcionando;



5. semeia os seis cargos;

6. cria o usuário Dono com o cargo Dono, sem senha utilizável, e gera o convite.

Roda **assíncrono**: 29 migrations levam segundos demais para um request HTTP. A tela de aprovação acompanha por *polling*; falha descarta o schema parcial. No boot, um bean explícito roda o Flyway do `platform` e depois o de cada cliente ativo, para que um deploy novo migre todas as empresas antes do primeiro request.

## 5. Fase 3 — Roteamento de schema por requisição

É o coração do isolamento, e cabe em três classes.

| Peça                               | Responsabilidade                                                                            |
|------------------------------------|---------------------------------------------------------------------------------------------|
| <code>CurrentTenant</code>         | Guarda o schema da requisição em curso.                                                     |
| <code>TenantAwareDataSource</code> | Ao entregar uma conexão, executa <code>SET search_path TO "&lt;schema&gt;", public</code> . |
| <code>TenantFilter</code>          | Resolve qual é o tenant e valida que o host bate com o token.                               |

### Por que no DataSource, e não no Hibernate

O Hibernate tem suporte nativo a multi-tenancy por schema, mas ele **não cobriria este projeto**: a maior parte das consultas usa `JdbcClient` cru, que passa por fora dele. Fazer a troca no nível do `DataSource` cobre JPA e `JdbcClient` de uma vez só.

#### Três cuidados que o código precisa ter, e que os testes vão cobrir:

- 1. Injeção.** `search_path` não aceita *bind parameter*. O nome do schema **nunca** vem de string do request: vem de um registro em memória alimentado por `platform.tenants`. A lista de permitidos é a proteção.
- 2. Vazamento entre clientes.** Conexão de pool é reaproveitada. O `search_path` é reescrito em **todo checkout**, sem depender de reset no fechamento — assim uma conexão devolvida nunca carrega o schema do cliente anterior.
- 3. Ordem dos filtros.** O `TenantFilter` precisa rodar **antes** da cadeia de segurança, porque a Fase 4 resolve permissões consultando a tabela de cargos dentro do schema do cliente. Se a ordem inverter, a consulta cai no schema errado e todo mundo perde acesso.

### Ordem de resolução do tenant

- 1. JWT autenticado** → a empresa do token. Continua sendo a fonte de verdade.
- 2. Rota pública** (login, convite) → slug extraído do host.
- 3. Token de plataforma + cabeçalho de suporte** → o caso da Fase 7.

Token de um cliente usado no subdomínio de outro devolve **403**. Sem tenant no contexto, o `search_path` aponta para `platform`.

## 6. Fase 4 — Autorização por permissão (o enum morre)

Com cargos livres, "gestor" deixa de significar a mesma coisa em duas transportadoras. A autorização não pode mais perguntar *qual é o seu cargo* — tem que perguntar *o que você pode fazer*.

| Antes                                                            | Depois                                           |
|------------------------------------------------------------------|--------------------------------------------------|
| <code>hasRole('OWNER') or hasRole('MANAGER')</code>              | <code>hasAuthority('vehicles.create')</code>     |
| <code>... or hasRole('OPERATOR')</code> (edição de cadastro)     | <code>hasAuthority('vehicles.update')</code>     |
| <code>hasRole('OWNER') or hasRole('SUPER_ADMIN')</code><br>(MiX) | <code>hasAuthority('integrations.manage')</code> |
| Checagem solta em <code>FleetContextBuilder</code>               | consulta à lista de permissões                   |

São as 19 anotações dos três controllers mais aquela checagem — conversão mecânica.

### De onde vêm as permissões

O token carrega o **identificador do cargo**, não a lista de permissões. Um catálogo em memória, chaveado por empresa e cargo e invalidado a cada edição, resolve as permissões por requisição.

**Por que não colocar as permissões dentro do token.** O token de acesso dura 60 minutos por decisão já registrada na configuração, e não é revogável. Com as permissões embutidas, tirar o acesso de alguém só teria efeito até uma hora depois — exatamente o tipo de coisa que vira chamado de suporte: *"removi o acesso dele e ele continua entrando"*. Resolvendo por requisição, a mudança vale na hora seguinte.

### Gestão de cargos

Endpoints de listar, criar, editar e apagar cargos, mais o catálogo de permissões. Regras que o serviço impõe:

- cargo de sistema não é editável nem apagável;
- apagar cargo em uso exige informar para qual cargo migrar as pessoas;
- gestão de equipe e de cargos são rejeitadas se vierem no corpo — não são delegáveis;
- permissão fora do catálogo devolve erro;
- toda escrita invalida o catálogo em memória e grava na auditoria.

## 7. Fase 5 — Autenticação com dois mundos

---

O login passa a receber o slug, que o SPA deriva do endereço do navegador:

- `app` → procura na tabela de contas da plataforma; o token sai marcado como plataforma, sem empresa associada.
- Qualquer outro slug → resolve a transportadora, exige que ela esteja provisionada e ativa, e procura o usuário **dentro do schema do cliente**. O token ganha a empresa, o slug e o cargo.

A renovação de sessão passa a guardar o schema junto do token de refresh — hoje ela busca o usuário sem saber onde procurar.

### Bootstrap do primeiro administrador

No boot: se não houver nenhum TI Topo ativo e as variáveis de bootstrap estiverem definidas, cria um com troca de senha obrigatória. Idempotente, e nenhuma senha no repositório.

**Correção obrigatória de CORS.** Hoje a configuração usa uma lista fixa de origens, que **não aceita curinga** quando o envio de credenciais está ligado — e está. Com subdomínios por cliente, precisa virar padrão de origem ( `https://*.rookhub.com.br` ), que o Spring resolve por requisição e é compatível com credenciais. Sem isso, nenhum cliente novo consegue logar.

## 8. Fase 6 — Fluxo de acesso e ciclo de vida das contas

---

### Rotas públicas

| Rota                                                | O que faz                                            |
|-----------------------------------------------------|------------------------------------------------------|
| <code>POST /v1/public/access-requests</code>        | Grava a solicitação e enfileira o e-mail para o time |
| <code>GET /v1/public/invites/{token}</code>         | Valida e devolve empresa, e-mail e cargo — nada além |
| <code>POST /v1/public/invites/{token}/accept</code> | Define a senha, ativa o usuário e já emite a sessão  |

Com limite por IP e campo-armadilha contra robô. O contrato do primeiro precisa ser documentado para o time do site institucional.

### Aprovação

Valida o slug → cria a transportadora → dispara o provisionamento → cria o Dono e o convite → enfileira o e-mail → registra na auditoria → registra `<slug>.rookhub.com.br` como domínio do projeto Pages via API da Cloudflare.

**Métricas globais:** um serviço percorre as transportadoras ativas trocando o tenant corrente e agrega em Java. Suficiente na ordem de grandeza atual; se um dia pesar, vira uma consulta única — possível justamente porque é um Postgres só.

## Equipe da transportadora

Convidar, trocar cargo, reenviar convite e desligar. Reusa **o mesmo** mecanismo de convite do Dono — um caminho só, testado uma vez.

**Falha de segurança encontrada durante a análise.** O `GET /v1/team` hoje **não tem controle de acesso**: qualquer usuário autenticado — inclusive um operador — lista e-mail e cargo de todo o painel da empresa. Entra como correção nesta fase, independentemente da decisão sobre o resto do plano.

## Desligamento: desativar e anonimizar

O botão "excluir" **não** apaga a linha. Ele:

1. marca a conta como inativa, registrando quem desligou e quando;
2. **revoga os tokens de sessão na hora** — sem isso a pessoa seguiria logada por até 30 dias;
3. registra na auditoria.

Um job diário anonimiza quem passou do prazo de retenção: o nome vira "Usuário removido", o e-mail vira um endereço inválido e a senha é zerada.

**Por que não apagar de verdade.** As consultas do assistente, as leituras de notificação e as dispensas de alerta têm chave estrangeira para usuários. A seed de desenvolvimento documenta que um `DELETE` em usuários já **derrubou o boot da API** por causa dessa chave. Desativar preserva histórico e auditoria; anonimizar atende o direito de eliminação da LGPD. Na tela o botão continua se chamando "excluir" — o usuário não vê diferença.

**Guarda obrigatória:** não é possível desativar o último Dono ativo da transportadora.

## 9. Fase 6b — A aprovação parametriza o ambiente

A aprovação não gera só a credencial do Dono. O TI Topo parametriza três coisas, e a empresa só fica pronta quando as três estiverem resolvidas. A tela de aprovação vira um formulário de quatro passos: **dados e slug** → **telemetria** → **marca** → **plano**. O objetivo é que o Dono, no primeiro login, encontre a plataforma conectada à frota, com a marca dele e com os recursos contratados ativos.

### 1. Telemetria

Boa notícia: a arquitetura **já é multi-fornecedor** — há tabela de integrações por empresa, credenciais cifradas em AES-256-GCM e um registro que garante o vínculo. Falta só a parametrização no momento da aprovação. A má notícia é que **só a MiX tem conector implementado**.

| Escolha do TI Topo | O que acontece                                                                                                                                                                       |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MiX                | Grava a credencial cifrada com o mecanismo que já existe, testa a conexão e libera a coleta. A empresa nasce enxergando a própria frota.                                             |
| Outro fornecedor   | Registra o fornecedor atual do cliente como <b>pendente de conector</b> . <b>O ambiente é liberado assim mesmo</b> — o cliente usa cadastro e gestão enquanto o conector não existe. |
| Nenhum ainda       | A RookHub indica o fornecedor homologado e acompanha a contratação, que é feita <b>pelo cliente</b> .                                                                                |

**A conta no fornecedor é do cliente, não da RookHub.** Isso preserva o que a migration de integração multiempresa já documenta — *"cada transportadora contrata a MiX diretamente... a RookHub não é a revendedora do rastreamento"* — e evita que a saída de um cliente vire disputa sobre rastreador instalado. A RookHub providencia no sentido de indicar, homologar e integrar; não de faturar o rastreamento.

A tela de integrações precisa dizer **em que estado está** — "aguardando conector", "aguardando contratação", "coletando" — em vez de mostrar uma frota vazia sem explicação.

### 2. Marca

Fica no schema da plataforma, e não no do cliente, por um motivo prático: é lido **antes do login**, quando ainda não existe sessão nem empresa no token.

```
CREATE TABLE platform.tenant_branding (  
  tenant_id      UUID PRIMARY KEY REFERENCES platform.tenants (id) ON DELETE CASCADE,  
  logo_mime      TEXT,  
  logo_bytes     BYTEA,  
  color_primary  TEXT NOT NULL DEFAULT '#d5623a',  
  color_accent   TEXT,  
  font_family    TEXT NOT NULL DEFAULT 'default', -- chave da lista homologada
```

```
updated_by    UUID REFERENCES platform.platform_users (id),
updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT branding_color_check CHECK (color_primary ~ '^[0-9a-fA-F]{6}$')
);
```

Guardar a imagem em `BYTEA` segue o padrão já estabelecido para a foto do motorista, cuja decisão está documentada no projeto: o Object Storage da Oracle é o destino certo, mas a conta ainda está em teste. Migrar depois é trocar o corpo de duas rotas. Limite de 256 KB, PNG ou SVG.

**Por que nenhum componente precisa mudar.** A folha de estilo do projeto já define `--primary` e mapeia as cores dos componentes para essa variável. A marca do cliente é um bloco de estilo injetado no carregamento, sobrescrevendo as variáveis. A estrutura de telas e componentes permanece padronizada — exatamente como pedido — e a personalização não vira dívida de manutenção.

**A fonte vem de uma lista homologada pela RookHub**, não de arquivo enviado pelo cliente. Upload livre traria licenciamento de terceiro para dentro da nossa hospedagem e risco de uma tipografia ilegível em tela de operação.

Um endpoint público resolve a marca pelo endereço, de modo que a tela de login do subdomínio já apareça com a identidade da transportadora, antes de qualquer sessão.

### 3. Módulos por plano

**Hoje o plano não existe no servidor.** A definição de planos, módulos e limites vive apenas no navegador; o backend nunca verifica. Na prática, bastaria abrir o DevTools para usar um módulo não contratado. Enquanto isso não mudar, não há como cobrar por pacote.

A solução encaixa plano e cargo num ponto só. O catálogo de permissões ganha um mapa de *módulo* → *permissões*, e o plano passa a ser um **filtro sobre o catálogo**:

1. o plano da empresa define quais módulos existem;
2. o catálogo **intersecta** as permissões do cargo com as do plano;
3. o editor de cargos só oferece o que o plano cobre — o Dono nem vê o que não foi contratado;
4. rota de módulo não contratado devolve **402**, e não 403: a distinção permite à tela oferecer upgrade em vez de dizer "sem acesso".

O plano é o teto da empresa; o cargo distribui o que sobrou. Trocar o plano vale na requisição seguinte, pelo mesmo mecanismo de revogação imediata da Fase 4.

Os planos também definem **limite de veículos e de usuários**. Ficam **fora** deste plano de trabalho: entram quando houver cobrança de verdade. Hoje o convite não é barrado por número.

## 10. Fase 7 — Acesso de suporte aos dados do cliente

---

O TI Operacional precisa enxergar o problema para resolvê-lo. O mecanismo: um token de plataforma mais um cabeçalho indicando qual transportadora abrir.

**Isto contradiz frontalmente a regra de ouro da tenancy deste projeto**, escrita no próprio código: *"SEMPRE da claim do token, nunca de parâmetro, cabeçalho ou corpo"*. É uma exceção deliberada e precisa ficar documentada no próprio arquivo — senão, com o tempo, vira exatamente a brecha que aquele comentário existe para evitar.

O que a torna segura, e os quatro pontos devem ser tratados como requisito, não como detalhe:

1. o cabeçalho **só** é lido quando o token é de plataforma; num token de cliente ele é ignorado;
2. **qualquer método diferente de leitura é recusado**, no filtro, antes do controller;
3. toda requisição grava na auditoria: quem, quando, qual empresa, qual rota;
4. o backoffice tem uma tela de auditoria de acessos de suporte, visível ao TI Topo.

Se mais adiante a equipe quiser "entrar como o cliente" (impersonação), é aqui que encaixa — mas exige consentimento do Dono, prazo de validade e trilha própria. Está fora deste plano.

## 11. Fase 8 — E-mail (Resend)

---

Novo pacote seguindo o padrão de cliente HTTP que já existe na integração com a MiX: um enviado, uma fila com tentativas e *backoff*, e modelos em HTML embutido — sem dependência nova.

Cinco modelos: solicitação recebida (interno), convite do Dono, convite de membro, solicitação recusada e conta desligada. Em desenvolvimento o envio fica desligado e grava no log.

**Pré-requisito externo:** verificar o domínio `rookhub.com.br` na Resend, o que exige registros DNS na Cloudflare — que vocês já administram.

## 12. Fase 9 — Frontend

---

Um módulo novo extrai o slug do endereço; `app` significa modo plataforma. O slug entra no login e num cabeçalho de todo request.

**O mapa estático de permissões morre.** Hoje a sessão é montada no navegador a partir de uma tabela fixa que traduz cargo em permissões. Passa a vir do servidor. São 50 usos em 19 arquivos, mas quase todos já passam por `hasPermission()` — a abstração certa já existe, muda só quem a alimenta.



| Arquivo                                                 | Mudança                                                                                                                |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>app/permissions.ts</code>                         | o mapa cargo → permissões e os rótulos fixos saem; sobram os helpers sobre a lista do servidor                         |
| <code>types/auth.ts</code>                              | o cargo vira objeto (id, nome, tela inicial); a sessão ganha escopo e slug                                             |
| <code>app/router.tsx</code>                             | a tela inicial vem do cargo; a área administrativa exige escopo de plataforma; guarda nova valida o slug contra o host |
| <code>management/features/roles/</code>                 | <b>novo</b> — lista de cargos, editor com checkboxes por módulo, exclusão com migração de usuários                     |
| <code>management/features/team/api.ts</code>            | trocar os dados simulados por convidar, trocar cargo e desligar de verdade                                             |
| <code>pages/saas/saas-requests-page.tsx</code>          | <b>nova</b> — fila de solicitações, aprovar/recusar, definir slug e plano                                              |
| <code>pages/saas/saas-team-page.tsx</code>              | <b>nova</b> — contas da equipe RookHub, só para o TI Topo                                                              |
| <code>pages/saas/saas-audit-page.tsx</code>             | desmocar; inclui os acessos de suporte                                                                                 |
| <code>pages/saas/saas-tenants-page.tsx</code> e detalhe | trocar os dados simulados pela API real                                                                                |
| <code>pages/login/login-page.tsx</code>                 | substituir o login simulado como gestor pelo aceite real do convite                                                    |
| <code>pages/misc/change-password-page.tsx</code>        | <b>nova</b> — primeiro acesso                                                                                          |
| <code>components/layout/demo-controls.tsx</code>        | o seletor de perfil da demo assume cargos dinâmicos ou fica restrito aos mocks                                         |

## 13. Fase 10 — Infraestrutura e publicação

| Item                                 | O que muda                                                                                                                                                |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| DNS                                  | <code>*.rookhub.com.br</code> apontando para o projeto Pages. O SSL universal da Cloudflare já cobre um nível de subdomínio.                              |
| Cloudflare Pages                     | O plano gratuito não aceita domínio curinga. Cada aprovação registra o domínio do cliente via API. Limite de 100 domínios por projeto — folgado por anos. |
| <code>docker-compose.prod.yml</code> | origens de CORS com curinga; entram a chave da Resend, as variáveis de bootstrap e as três da Cloudflare.                                                 |
| <code>deploy/Caddyfile</code>        | Sem mudança — a API continua só em <code>api.rookhub.com.br</code> .                                                                                      |
| TimescaleDB                          | Cada schema ganha suas próprias <i>continuous aggregates</i> . Subir o limite de workers de segundo plano conforme o número de clientes cresce.           |
| Backup                               | O dump do banco inteiro já pega todos os schemas. Documentar o restore de <b>um</b> cliente, que passa a ser possível isoladamente.                       |

### Migração dos dados atuais

Script único, rodado uma vez com a API parada. Ele move todas as tabelas e views de `public` para `tenant_servioeste` — incluindo o próprio histórico do Flyway. Levar o histórico junto é o que faz o Flyway do tenant enxergar as 27 migrations como já aplicadas e rodar só as duas novas. Nenhuma telemetria coletada se perde.

## 14. Como isso é verificado

### Automatizado

- **Vazamento de schema:** conexão reaproveitada do pool não carrega o schema do request anterior.
- **Provisionamento:** schema criado, 29 migrations aplicadas, 6 cargos semeados, desfazimento limpo em falha.
- **Cargos:** cargo de sistema não é editável nem apagável; permissão não delegável no corpo é rejeitada; apagar cargo em uso sem destino é recusado.
- **Revogação imediata:** tirar uma permissão do cargo tem efeito na **requisição seguinte**, sem esperar o token expirar.
- **Autenticação:** o mesmo e-mail em duas empresas loga em cada uma com a sua senha; token de um cliente no host de outro é recusado.
- **Suporte:** leitura com o cabeçalho aparece na auditoria; escrita no mesmo caminho é recusada; token de cliente com o cabeçalho o ignora.

- **Desligamento:** revoga a sessão na hora; o último Dono não pode ser desativado; a anonimização preserva o histórico do assistente e das notificações.
- **Planos:** rota de módulo fora do plano devolve 402; subir o plano libera na requisição seguinte; o editor de cargos não oferece o que não foi contratado.
- **Marca:** a marca é servida sem sessão; logo acima do limite, cor mal formada ou fonte fora da lista são recusados.
- **Telemetria:** aprovar com MiX grava a credencial cifrada e testa a conexão; aprovar com outro fornecedor deixa pendente e **ainda assim** libera o ambiente.

## Manual, ponta a ponta

1. Enviar um pedido de acesso pela rota pública e conferir a linha gravada e o e-mail no log.
2. Entrar como TI Topo e aprovar com o slug `amazonas`, escolhendo fornecedor "outro", logo e cor próprios, e plano **Starter**.
3. Conferir no banco: o schema existe, tem os 6 cargos semeados, e a tabela de usuários contém **só** o Dono, ainda inativo.
4. **Abrir o subdomínio sem login:** a tela de login já mostra o logo e a cor da Amazonas.
5. Abrir o convite, definir senha e entrar. A tela de integrações diz "aguardando conector", e não uma frota vazia sem explicação.
6. **Confirmar que Analytics e IA não aparecem** (fora do Starter) e que chamar a rota direto devolve 402. Subir para Business no backoffice e confirmar que aparecem **sem novo login**.
7. **Como Dono, criar o cargo "Diretor Financeiro"** com visão de análise e cobrança, convidar alguém nele, aceitar, e confirmar que essa pessoa não vê veículos nem consegue editar.
8. **Tirar a permissão de cobrança do cargo e confirmar que a tela some sem novo login.**
9. Tentar desativar o Dono → recusado. Desativar o Diretor → ele cai na próxima requisição.
10. Como TI Operacional, abrir a Amazonas em leitura; tentar uma escrita → recusada; conferir as duas linhas na auditoria.
11. **Prova de isolamento:** logar na Servioeste, chamar uma rota de frota e conferir no log do Postgres que a conexão está no schema certo. Repetir na Amazonas.

## Em produção

Rodar o script de migração com a API parada, subir com as variáveis de bootstrap definidas, logar em `app.rookhub.com.br`, trocar a senha e então **remover as variáveis de bootstrap** do ambiente e reiniciar.

## 15. Ordem de execução

| Ordem | Fases                                                     | Observação                                                                       |
|-------|-----------------------------------------------------------|----------------------------------------------------------------------------------|
| 1º    | 1, 2 e 3 — plataforma, cargos, roteamento de schema       | Base. Não entrega tela nenhuma, e nada depois funciona sem ela.                  |
| 2º    | 4 — autorização por permissão                             | Mecânico, mas toca 19 anotações e a sessão do frontend. Melhor fazer de uma vez. |
| 3º    | 5 — autenticação e bootstrap                              | Primeiro momento em que é possível logar em produção.                            |
| 4º    | 6, 6b, 7 e 8 — endpoints, parametrização, suporte, e-mail | O fluxo passa a existir ponta a ponta, ainda sem telas.                          |
| 5º    | 9 — telas                                                 | Desmocar backoffice, convite e equipe; editor de cargos.                         |
| 6º    | 10 — infra                                                | As variáveis acompanham a fase 5; DNS e domínios ficam para o fim.               |

### Pontos que a equipe precisa decidir

- **Cargos customizáveis aposentam o enum de perfis.** É a mudança de maior alcance do plano e não gera tela nova até a fase 9. Vale confirmar o apetite antes de começar.
- A migração para schema por tenant é a parte mais pesada e também não gera tela. Fazer agora é bem mais barato que fazer com cinco clientes em produção.
- Reservar `app.rookhub.com.br` ao Super Admin **muda o endereço que a empresa parceira já usa hoje.** Precisa de aviso e, de preferência, redirecionamento temporário.
- O acesso de suporte por cabeçalho é uma exceção consciente à regra de ouro da tenancy. Confirmar se leitura auditada basta, ou se o time vai querer impersonação depois.
- Confirmar a suposição de que gestão de equipe e de cargos **não** são delegáveis pelo Dono a um Diretor.
- O registro de domínio na Cloudflare é um ponto de falha externo no meio da aprovação. Se falhar, a empresa fica provisionada mas inacessível — o backoffice precisa de um botão para tentar de novo.
- **Só a MiX tem conector.** Aceitar um cliente de outro fornecedor significa entregar um ambiente sem telemetria por tempo indeterminado. Decidir se isso se vende, e com qual promessa de prazo.
- **Os limites de veículos e usuários dos planos ficam de fora** deste plano de trabalho, apesar de já definidos no código. Entram quando houver cobrança. Confirmar se é aceitável.